HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

HPC - Projet - GEGL

Christophe Künzli (christophe.kunzli@heig-vd.ch)
Léonard Jouve (leonard.jouve@heig-vd.ch)

June 8, 2026

All our code is in this (repository)

Contents

1 Introduction

During this project, we were requested to analyze the performance of part of an open source project and propose some optimizations. We chose the GEGL project (repository), which is an image processing library used by GIMP. We focused on its transformation operations (see `operations/transform`), and more precisely on the `gegl_operation_transform` operation, which is used to perform one or multiple affine transformations on images, such as scaling, rotation, and shearing. We chose this operation because it is widely used in image processing, and it is a good candidate for optimization, as it involves a lot of computations.

2 Build project

GEGL uses Meson to build the project and manage dependencies.

All listed dependencies can be found in the root `meson.build` file. Most of them can be installed via your platform package manager.

One of the dependencies is `babl` (repository). It is another GNOME project used as a pixel encoding and color space conversion engine.

It can be installed by following the `INSTALL.in` instructions.

Each dependency must include a `.pc` file in `PKG_CONFIG_PATH` to be found by Meson.

```
meson setup buildDir
meson compile -C buildDir -j6
```

3 Benchmark

We wrote a simple benchmark that runs a transform operation 1000 times and computes the average time spent on the operation, along with the minimum and maximum times.

How to run the benchmark

```
# Run all tests
meson test -C buildDir

# Run all benchmarks
meson test -C buildDir --benchmark

# Run only our benchmark
meson test -C buildDir --benchmark 'Perf Test transform'
```

Important notes.

- You can switch between the original implementation and our implementation by opening `operations/transform/transform-core.c` and commenting or uncommenting `#define TRANSFORM_HALIDE`, then rebuilding the project. If `TRANSFORM_HALIDE` is defined, the benchmark uses our implementation; otherwise, it uses the original implementation.
- Text sent to standard output during the benchmark is not shown in the terminal when running the benchmark, because Meson captures benchmark output. To see the output, check the log file generated by Meson in `buildDir/meson-logs/testlog.txt`.
- The benchmark can also be used to perform a predefined transformation on an actual image and save it to disk. To use this feature, open `perf/test-transform.c` and set `IMAGE_IN` and `IMAGE_OUT` to the input and output image paths respectively. Then rebuild the project and run the benchmark as shown above. The output image will be saved to the specified path.

4 Analysis

Our first step was to analyze the code of the `gegl_operation_transform` operation, and identify the most computationally expensive parts. We wrote a simple benchmark in order to measure the time of a simple image transformation pipeline and also generated a flame graph to find bottlenecks using VTune.

4.1 Flame graph

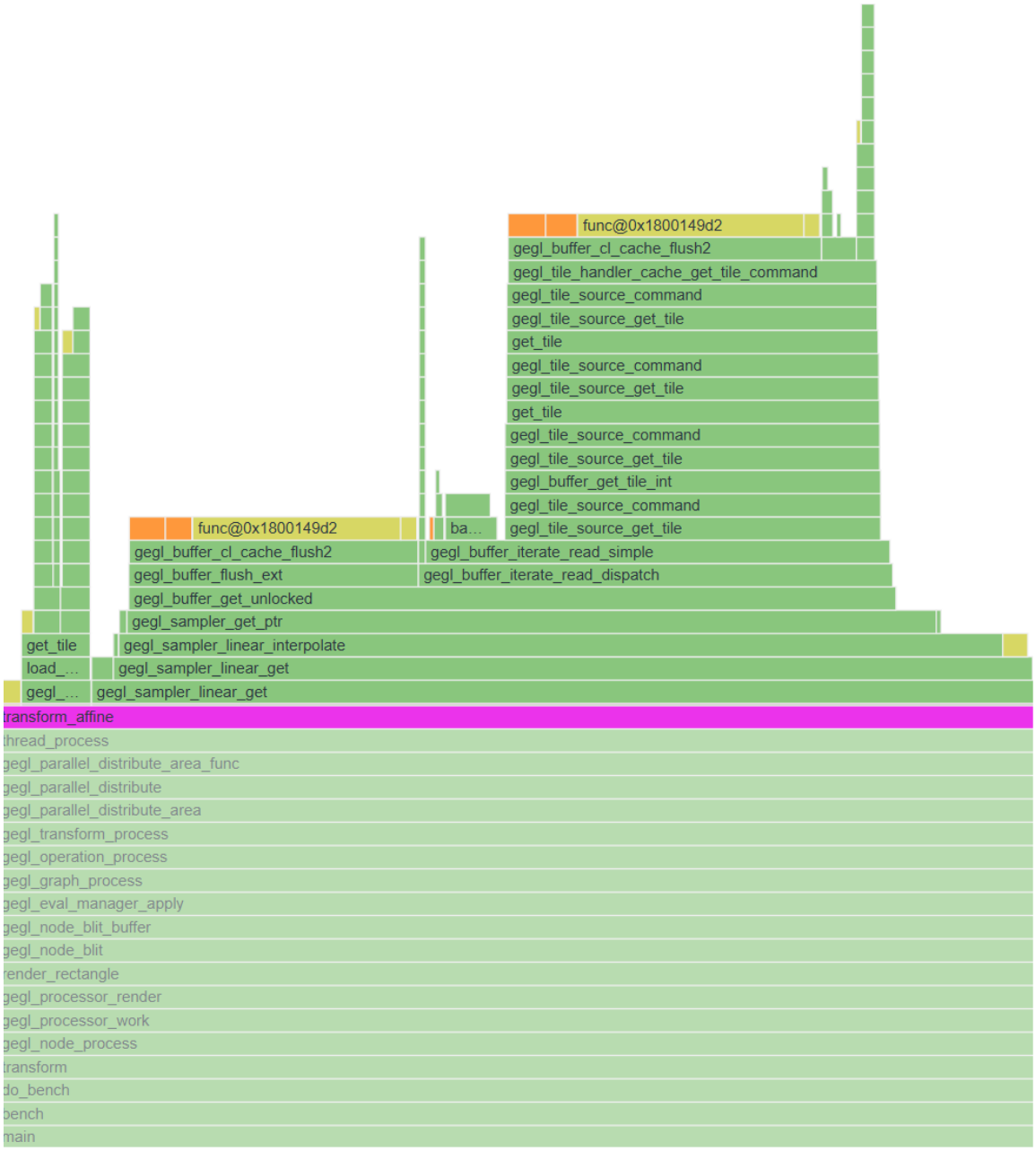

Figure 1: Flame graph

As we can see, `transform_affine` takes almost 100% of the `main` function time. This is not surprising as we are only doing an affine transformation. We also see there is a big overhead for a simple operation such as affine transformation.

5 Affine transformation

5.1 Overview

Image transformations are fundamental operations in image processing and computer graphics. They modify the spatial arrangement of pixels to achieve effects such as translation, rotation, scaling, and shearing.

Affine transformations form an important class of geometric transformations because they preserve straight lines and parallelism. As a result, they can represent a wide range of practical image manipulations while remaining computationally efficient.

5.2 Transformation matrices

Affine transformations are typically represented using matrices. A transformation matrix encodes how the coordinates of a pixel should be modified when applying a transformation.

For example:

- a translation matrix moves pixels horizontally and vertically,
- a scaling matrix enlarges or shrinks an image,
- a rotation matrix rotates pixels around a specified point,
- a shear matrix skews the image along one axis.

One of the main advantages of matrix-based transformations is that multiple operations can be combined into a single matrix. For example, an image can be scaled, rotated, and translated by multiplying the corresponding matrices together and applying the resulting matrix once.

5.3 Forward mapping

A straightforward way to transform an image is to iterate through all source pixels and compute where they should appear in the destination image.

- Read a pixel from the source image.
- Apply the transformation matrix to its coordinates.
- Write the pixel to the resulting position in the destination image.

This approach is called forward mapping. While simple, forward mapping has a major drawback: some destination pixels may never receive a value, creating visible holes or gaps in the transformed image.

5.4 Inverse mapping

To avoid gaps, most image processing libraries use inverse mapping.

For each pixel in the destination image:

- Apply the inverse transformation matrix.
- Compute the corresponding position in the source image.
- Read the source pixel value.
- Write the value to the destination pixel.

Using inverse mapping guarantees that every pixel in the output image is assigned a value.

5.5 Advantages of matrix-based transformations

Representing transformations as matrices offers several advantages:

- Multiple operations can be combined into a single matrix.
- It is well suited to optimization and parallelization.

Consequently, matrix-based affine transformations are the standard technique used in modern image processing libraries and graphics frameworks.

See this resource: YouTube video on affine transformations.

6 Proposed optimization

We tried to reduce the overhead of the operation by using a more efficient algorithm for affine transformation with Halide. We only focused on affine transformations as a proof of concept, but this could be generalized to all transformations in future work.

6.1 Halide

Halide (official website) is a domain-specific language for image processing and computational photography. It is designed to make it easier to write high-performance image processing code, and we used it to see if it performs better than the current implementation of `gegl_operation_transform`.

We used Halide version 21.0.0.

See also: Halide tutorials.

6.2 Implementation

See `operations/transform/halide.cpp` and `operations/transform/transform-core.c`.

Our implementation replaces the function `transform_affine`. We can switch between our implementation and the original implementation by defining or not defining `TRANSFORM_HALIDE` in `operations/transform/transform-core.c`.

`halide.cpp` is built as a standalone object file and header file. These object and header files are linked to the GEGL library.

Simplified Halide code

```
ImageParam input(UInt(8), 3, "input");
ImageParam inverse_matrix(Float(32), 2, "matrix");
Param<int> roi_x{"roi_x"};
Param<int> roi_y{"roi_y"};

Var x, y, color;

Func transform;

Expr xf = cast<float>(x + roi_x) + 0.5f;
Expr yf = cast<float>(y + roi_y) + 0.5f;

Expr u =
    inverse_matrix(0, 0) * xf +
    inverse_matrix(1, 0) * yf +
    inverse_matrix(2, 0);
Expr v =
    inverse_matrix(0, 1) * xf +
    inverse_matrix(1, 1) * yf +
    inverse_matrix(2, 1);

Expr sx = clamp(cast<int>(floor(u)), 0, input.dim(1).extent() - 1);
Expr sy = clamp(cast<int>(floor(v)), 0, input.dim(2).extent() - 1);

transform(color, x, y) = input(color, sx, sy);
```

An important thing to note is that our implementation does not properly support all transform operations. For example, shearing and scaling are not supported in our version. Also, we only focused on RGB image format (no alpha) and did not test other formats, so our implementation may not work properly with other formats.

Due to time constraints, we were unable to identify the root cause of the bug. However, this project is not intended to be a drop-in replacement for the current implementation. Instead, it serves as a proof of concept to evaluate whether the idea is worth pursuing further.

6.3 Comparison with current implementation

Implementation	Time (s)	Average (s)	Min (s)	Max (s)
Original	20.351498	0.020351	0.014517	0.035250
Halide	3.693380	0.003693	0.002921	0.006582

While this is a very simple benchmark, it shows that our implementation using Halide is significantly faster than the original implementation. The average time is reduced by a factor of 5.5, and the minimum and maximum times are also significantly reduced.

The results are somewhat biased because our Halide solution is simplified compared to the existing one and does not act as a full replacement, but they are very encouraging.

6.4 Further improvements and optimizations

- Implement support for all transform operations, including shearing and scaling.
- Add support for more image formats, including those with alpha channels.
- Optimize the Halide implementation further by exploring scheduling strategies and parallelization techniques offered by Halide.

7 Difficulties encountered

- Understanding and building the GEGL project.
- Understanding affine transformations and how they are implemented in the current codebase.
- Learning how to use Halide and how to integrate it into the existing codebase.

8 Conclusion

Benchmark results show a significant performance improvement compared to the current implementation. The Halide-based version achieved an average execution time approximately 5.5 times lower than the original implementation, demonstrating the potential benefits of using a DSL designed for image processing.

However, the current implementation should not be considered a drop-in replacement for the existing GEGL transform operation. Several features supported by the original implementation are not yet implemented, and the benchmark was performed on a simplified workload. Consequently, the results should be interpreted as an indication of the potential performance gains rather than a direct comparison between two feature-equivalent implementations.

Despite these limitations, the project successfully demonstrates the feasibility of integrating Halide into GEGL for affine image transformations. Future work could focus on supporting the complete set of transformation operations, improving compatibility with existing image formats, and exploring additional optimization opportunities through Halide's scheduling and parallelization capabilities.

Overall, the results are encouraging and suggest that further investigation of a Halide-based implementation is justified.